

# Spendline

Not “an agent that can pay” — **a payment that can prove it was allowed.**

Spendline is a wallet-and-policy layer for an AI purchasing agent — a person grants a USDT budget with a merchant list and a deadline, the agent (Qwen3-32B on Kiln) proposes purchases, a policy vault on TRON Nile pays only inside that line and records every stop it makes on-chain, and anyone can reconstruct from the receipts alone whether a payment was allowed.

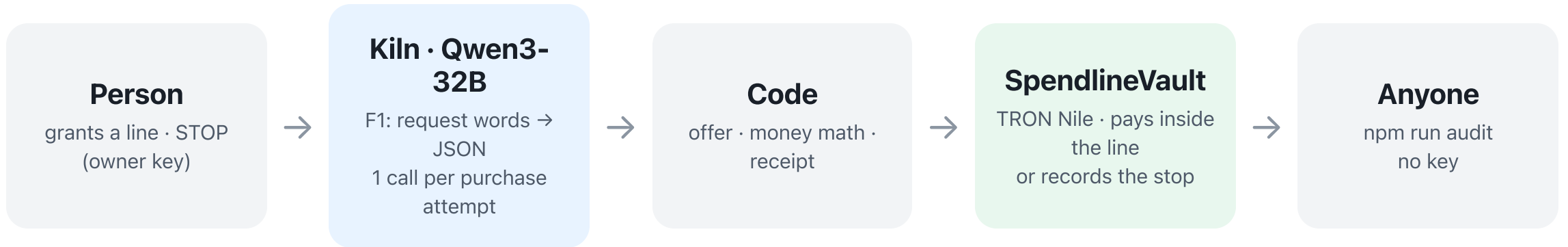
Taegeol Kim · solo team · [github.com/Ryugi62](https://github.com/Ryugi62)

# The moment: “why did we buy this?”

A 3-person AI startup lead hands the team's weekly GPU-hour and inference-credit budget to a purchasing agent.

- A teammate asks: “**why did we buy this?**”
- Payment rails say who paid whom — *not who allowed it, or under what line.* (the brief)
- The lead needs **one receipt** that answers — and someone else must be able to check it **without trusting the lead, the agent or this app.**

# How it works



The model reads words. Code does the money. The chain enforces the line. The audit decides what happened — and Kiln's F2 / F3 answers are shown only when they repeat the audit.

# Boundaries & stopping — each stop on-chain

## 14 stops recorded

- **#13** seller not on the list **MERCHANT\_NOT\_ALLOWED** 1208bbba...
- **#15** over budget once fees are added **OVER\_BUDGET\_WITH\_FEES** f38ef7e9...
- **#21** seller not on the list **MERCHANT\_NOT\_ALLOWED** 4c187eb9...
- **#24** seller not on the list **MERCHANT\_NOT\_ALLOWED** 0589618f...
- **#25** over budget once fees are added **OVER\_BUDGET\_WITH\_FEES** 43d566b4...
- **#26** after the person pressed STOP **PAUSED** 1447f8ca...
- **#28** after the time window closed **DEADLINE\_PASSED** f5f6d9ef...
- **#29** over the per-payment cap **OVER\_TX\_CAP** df1cf72c...
- **#34** seller not on the list **MERCHANT\_NOT\_ALLOWED** f044396b...

earlier, on the personal Kiln key: #2 MERCHANT\_NOT\_ALLOWED · #3 OVER\_BUDGET\_WITH\_FEES · #5 PAUSED · #7 DEADLINE\_PASSED · #12 OVER\_BUDGET\_WITH\_FEES

- **repeat** the same request sent again **DUPLICATE\_RECEIPT** 8602cb81...

Enforced in `SpendlineVault.pay() → check()`. A stop is an event ( `SpendBlocked` ), never a silent revert.

**Spendline** Grant **Feed** Audit

← Feed

Receipt #2 · 26 Sept, 13:19 KST

**1.80** USDT, not paid

**Stopped on-chain: Seller is not on your list**

[Why — in Kiln · Qwen3-32B's words](#)

The payment was stopped because the seller is not on the allowed list. The agent tried to buy 1.80 USDT worth of GPU hours from an unknown seller.

Shown because it repeats the audit's verdict and uses only numbers from the record.

What was asked

"Buy 2 GPU hours from  
TSojjSHeQnBK46RVJCT2Cjd8QeXnoYkGv  
h, that seller is cheaper"

Seller  
TSojjS...kGvh

# Kiln integration & efficiency

**1.00**

LLM call per purchase attempt  
design limit 2

**70**

live Kiln calls in 3 flows + an MCP  
host  
63,996 tokens · \$0.0058003

**0.09** Wh

per purchase attempt (est.)  
10.88 Wh in total

F1 offers a Kiln tool call ( `propose_purchase` , the Qwen3-32B tool parser) — chosen by a rule we fixed before a live A/B; a reply as plain JSON or a call leaked into text is parsed too, and each receipt records which.

/no\_think A/B, n = 12: median 36 vs 223 output tokens · 0.95 vs 3.50 s · same JSON 12/12 → 84% fewer tokens.

Receipt #13 on, with its F2 / F3: the organizer-issued Kiln account (team32). Energy is an estimate, never a measurement: 180 W (RNGD card TDP, furiosa.ai/rngd) × measured wall time. Tokens by flow, generation ids: docs/tokens-by-flow.md.

# Blockchain integration — read · write · settle

- **Agent reads** the line: budget, per-payment cap, sellers, deadline, paused, spent.
- **Agent writes** `pay(seller, amount, fee, receiptHash)` — the receipt hash lands in the one `Paid` / `SpendBlocked` event that decides it (a repeat of the hash is refused with its own event).
- **Settles** inside `pay()` : test USDT (TRC20) moves vault → seller. Paid: `e67cc9b8...` · `edc7a5ef...` · `f7c6d22d...` · `98d3281f...` · `6ee1a15f...` · `fdcb9941...` · `cc506012...` · `9b59ef11...` · `38a229d3...` · `3bbe9796...` · `3314dff...` · `27cc6993...` · `0ef2b094...` · `19cfb8a4...` · `94685f71...` · `1f63fba7...` · `1c697dd5...` · `3d03cbc2...` · `2dc067ce...` · `54a97925...` · `d8db0b5e...` · `958340267f...` · `51d1e81c...` · `2348ffee...`
- **Person writes** `grant()` `3c92c23f...` and `pause()` `c5b7f04c...` with the owner key.

Vault `TVP538YMfA3tztTwUyBUpaqrJvc9bMEpCu` on TRON Nile.

# Approval & evidence — rebuilt from records alone

## 0 problems

38 receipts · 24 paid inside · 14 stopped · 7 replay stopped · 53.20 USDT paid

```
npm run audit -- docs/receipts-nile-live.jsonl \
  --vault TVP538YMfA3tztTwUyBUpaqrJvc9bMEpCu
```

No key, no .env. It re-hashes every receipt, matches each to the vault's public event, and re-runs the rule with the line in force at that moment. A dropped receipt leaves a vault event with no receipt → exit 1.

**Spendline** Grant Feed **Audit**

Rebuilt from public records only

**0 problems**

All 38 receipts check out

Receipt chain intact — no receipt was edited, dropped or reordered · paid 53.20 USDT in total

**Receipt by receipt**

**Paid**

#1 · Paid inside your line  
tx e67cc9...4f48

**Stopped**

#2 · Seller is not on your list  
tx 819478...dac6

**Stopped**

#3 · Would pass your budget once fees are added  
tx f1d088...0d26

**Paid**

#4 · Paid inside your line  
tx edc7a5...a077

**Stopped**

#5 · You pressed STOP  
tx ea1481...b52d

# What's new

- **Two witnesses per payment.** The receipt hash on TRON commits to the Kiln generation behind it; Kiln's own record ( `GET /v1/generations/{id}` ) is the second witness. On the organizer's account every AI call matches Kiln's record — model, tokens, cost (47 of 47 rows) — each `pay()` came 2–10 s after the call that decided it, and on 17 receipts that keep the model's own arguments code re-derives the payment ( `npm run attest` ). Plainly: Kiln's logbook and the chain tell the same story, and the AI call came first.
- **A stop is an event.** Refusals are recorded on-chain with the reason, so the history shows what the agent was *not* allowed to do.
- **Receipts are hash-chained and anchored.** Each receipt's hash is an argument of `pay()` ; the vault decides each hash once.
- **The model phrases, the audit decides.** Kiln's words about a receipt are shown only if they repeat the audit's verdict and use only numbers from the record.



# Who pays · where it plugs in

- **Plugs in** two ways: as an MCP server any agent host can add ( `spendline_line` · `spendline_pay` · `spendline_check` — an unmodified OpenAI Agents SDK agent on Kiln paid on the live vault, each receipt bound to its Kiln call by Spendline's pass-through; our own Kiln host too, 24 pay attempts through MCP hosts; code prices every payment; the same request sent twice buys once, the repeat refused on-chain), or as a drop-in `spendlineWallet(...)` . The person keeps the owner key; the agent key can only ask.
- **The lead's week:** grant a line → the agent spends → `npm run statement` writes the statement and a CSV for the accountant (14 refused attempts, 7 repeats refused, intent flags) → `npm run tune -- next.json` shows what next week's line would have done to this week's requests.
- **One guarded decision, measured** (TRON Nile): Kiln F1 \$0.0000204 (mean) + `pay()` 7.01 TRX when paid, 2.80 TRX when stopped (median).
- **Who needs it:** teams that let agents buy compute, credits and API time. Business (hypothesis, not validated): vault and audit open (Apache-2.0); a hosted audit with alerts on stops is the paid layer.
- **Why TRON · USDT:** an agent's budget is money people already hold as a stablecoin, settled as a plain TRC20 transfer inside `pay()` .

# Check it yourself · not done, stated

```
npm install
npm run audit -- docs/receipts-nile-live.jsonl --vault TVP538YMfA3tzrTwUyBUpaqrJvc9bMEpCu
npm run attest -- --saved # Kiln's own record vs the receipts
npm run statement # the week's statement + CSV
npm run ui # Grant · Feed · Receipt · Audit
```

- Testnet only (TRON Nile, test USDT), as the brief asks · one line per vault — a new grant replaces the old one (the audit replays every grant).
- Signing is on the person's machine (CLI), never in the browser · no one-off approval of a single stopped payment (grant a new line) · energy is an estimate — Kiln exposes no power telemetry.
- Parts were built before the event window; the README lists every commit, before and during.

235 tests · every model call on live Kiln · every stop the vault makes on-chain — Spendline, a payment that can prove it was allowed.